Then by adding and substracting $\mathbb{E}[f(\overline{X}_N)]$, we get

$$|\mathbb{E}[f(X_T)] - \hat{\mu}_M| \leq \underbrace{\left|\mathbb{E}[f(X_T)] - \mathbb{E}[f(\overline{X}_N)]\right|}_{\text{discretization (bias)}} + \underbrace{\left|\mathbb{E}[f(\overline{X}_N)] - \hat{\mu}_M\right|}_{\text{integration/statistical error}}, \tag{251}$$

If we instead consider the MSE:

$$MSE = \mathbb{E}\left[\left(\frac{1}{M}\sum_{i=1}^{M} f\left(\overline{X}_N^{(i)}\right) - \mathbb{E}[f(X_T)]\right)^2\right]. \tag{252}$$

Again by adding and substracting $\mathbb{E}[f(\overline{X}_N)]$, we obtain

$$\text{MSE} = \underbrace{\left(\mathbb{E}[f(\overline{X}_N)] - \mathbb{E}[f(X_T)]\right)^2}_{\text{bias}^2} + \underbrace{\text{Var}\left(\frac{1}{M}\sum_{i=1}^{M} f\left(\overline{X}_N^{(i)}\right)\right)}_{\text{variance term}}. \tag{253}$$

8 Monte Carlo and Multilevel Monte Carlo

8.1 Monte Carlo Method

Monte Carlo Method estimates $\mathbb{E}[X]$ as a mean of M independent samples values $X(\omega)$ in a probability space $(\Omega, \mathscr{F}, \mathbb{P})$,

$$\bar{X}_M = \frac{1}{M}\sum_{i=1}^{M} X(\omega^{(i)}) \tag{254}$$

where $\mathbb{E}[\bar{X}_M] = \mathbb{E}[X] := m_X$. The variance of $\bar{X}_M$ (or the Mean Square Error of $\epsilon_M = m_X - \bar{X}_M$, see Proposition 5) is

$$\text{Var}(\bar{X}_M) = \mathbb{E}[(m_X - \bar{X}_M)^2] = \frac{\sigma_X^2}{M} \tag{255}$$

where $\sigma_X^2 = \text{Var}(X)$. So, the Root Mean Square error is

$$\mathbb{E}[(m_X - \bar{X}_M)^2]^{1/2} = \frac{\sigma_X}{\sqrt{M}} \tag{256}$$

An accuracy of ε requires (see Remark 26)

$$M = O\left(\frac{1}{\varepsilon^2}\right) \tag{257}$$

samples.

- Weakness of MC comes from (257), where the computational cost of MC can be high when each sample $X(\omega)$ comes from an approximate solution of a PDE or a discretization scheme.

Variance Reduction

We would like to improve the constant factor in the error (256) by reducing σ_X^2. Suppose there is a r.v Y such that $\mathbb{E}[X] = \mathbb{E}[Y]$, but with smaller variance, i.e.,

$$\sigma_Y^2 < \sigma_X^2$$

Then inserting σ_Y^2 in (256) will decrease the computational work to achieve a target error, provided that simulating Y is not more expensive that generating samples from X. With this approach, the same error can be achieved with fewer samples.

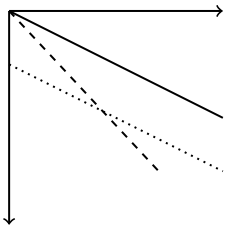

Figure 8: Improved convergence rate vs. improved constant.

- If we take log in both sides of (256) and plot $\log \sigma_X - \frac{1}{2} \log M$ in the x-axis against log error in the y-axis, we obtain the convergence rate of MC as a solid line with slope $-\frac{1}{2}$ in Figure 8.
- An improved constant will lead to a parallel shift of the line with slope $-\frac{1}{2}$, as the dotted line in Figure 8.
- An improved convergence rate would lead to a line with different slope, as in the dashed line with slope -1 in Figure 8.

Control Variates

Assume there exists a r.v. Y such that we know

$$\mathbb{E}[Y] \tag{258}$$

Then

$$\mathbb{E}[X] = \mathbb{E}\left[X - \lambda\left(Y - \mathbb{E}[Y]\right)\right] \tag{259}$$
$$= \mathbb{E}[X - \lambda Y] + \lambda \mathbb{E}[Y] \tag{260}$$

for any $\lambda \in \mathbb{R}$. Thus a Monte Carlo estimate for $\mathbb{E}[X]$ is given by

$$\bar{X}_M^\lambda = \frac{1}{M} \sum_{i=1}^{M} (X_i - \lambda Y_i) + \lambda \mathbb{E}[Y] \tag{261}$$

Assume that the simulation of $\bar{X}_M^\lambda$ requires at most c times the computing time of $\bar{X}_M$, where $c > 1$. We are going to chose λ such that

$$\text{Var}\,(X - \lambda Y) \text{ is minimized} \tag{262}$$

We have that

$$\text{Var}(X - \lambda Y) = \text{Var}(X) - 2\lambda\,\text{Cov}(X, Y) + \lambda^2\,\text{Var}(Y), \tag{263}$$

is minimized by choosing

$$\lambda^* = \frac{\text{Cov}(f(X), g(Y))}{\text{Var}(g(Y))} = \rho_{XY}\frac{\sigma_X}{\sigma_Y}, \quad \rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X\sigma_Y} \tag{264}$$

For λ^* and from (255), the MSE of $\bar{X}_M^{\lambda^*}$ is given by

$$\text{Var}(\bar{X}_M^{\lambda^*}) = \frac{\sigma_X^2}{M}(1 - \rho^2) \leq \frac{\sigma_X^2}{M} = \text{Var}(\bar{X}_M) \tag{265}$$

9 Multilevel Monte Carlo

Two Level Monte Carlo

Similarly as in variates control, We want to estimate $\mathbb{E}[P_1]$ but it is much cheaper to simulate P_0 which approximates P_1, then since

$$\mathbb{E}[P_1] = \mathbb{E}[P_0] + \mathbb{E}[P_1 - P_0] \tag{266}$$

we can use the unbiased two-level estimator

$$\frac{1}{N_0}\sum_{n=1}^{N_0} P_0^{(n)} + \frac{1}{N_1}\sum_{n=1}^{N_1}\left(P_1^{(n)} - P_0^{(n)}\right) \tag{267}$$

(a) We consider $P_1^{(n)} - P_0^{(n)}$ for the same underlying stochastic sample $\omega^{(n)}$, so that $P_1^{(n)}$ and $P_0^{(n)}$ are correlated and $P_1^{(n)} - P_0^{(n)}$ is small with small variance (see (263))

(b) The two main differences from the control variate approach is that $\mathbb{E}[P_0]$ is not known and $\lambda = 1$.

Let C_0 and C_1 be the cost of computing a single sample of P_0 and $P_1 - P_0$, respectively, then the total cost is

$$N_0C_0 + N_1C_1 \tag{268}$$

If V_0 and V_1 are the variance of P_0 and $P_1 - P_0$, then the overall variance is

$$\frac{1}{N_0}V_0 + \frac{1}{N_1}V_1 \tag{269}$$

Assuming that

$$\sum_{n=1}^{N_0} P_0^{(n)} \quad \text{and} \quad \sum_{n=1}^{N_1} \left(P_1^{(n)} - P_0^{(n)} \right) \tag{270}$$

use independent samples, we consider the optimization problem:

- **Objective:** Minimize V_{total} where

$$V_{\text{total}} = \frac{V_0}{N_0} + \frac{V_1}{N_1}$$

- **Constraint:**

$$N_0 C_0 + N_1 C_1 = C_{\text{Total}} \quad \text{(Total Cost, fixed)}$$

Hence, the variance is minimized for a fixed cost by choosing

$$\frac{N_1}{N_0} = \frac{\sqrt{\frac{V_1}{C_1}}}{\sqrt{\frac{V_0}{C_0}}} \tag{271}$$

9.1 Multilevel Monte Carlo

Given a sequence $P_0, \ldots, P_{L-1}$ which approximates P_L with increasing accuracy but also increasing cost, we have the simple identity

$$\mathbb{E}[P_L] = \mathbb{E}[P_0] + \sum_{\ell=1}^{L} \mathbb{E}[P_\ell - P_{\ell-1}] \tag{272}$$

and therefore we can use the unbiased estimator for $\mathbb{E}[P_L]$:

$$\underbrace{\frac{1}{N_0} \sum_{n=1}^{N_0} P_0^{(0,n)}}_{\text{level 0 estimate}} + \underbrace{\sum_{\ell=1}^{L} \frac{1}{N_\ell} \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)}_{\text{level } \ell \text{ ``correction''}} \tag{273}$$

- Within one level we reuse the *same* path to compute the pair $(P_\ell, P_{\ell-1})$ so that the *difference* benefits from strong coupling.
- Across levels you start afresh; level 2 does not reuse paths from level 1, etc. The superscript (ℓ, n) indicate that independent samples are used at each level.

Let C_0, V_0 to be the cost and variance of one sample of P_0, and C_ℓ, V_ℓ to be the cost and variance of one sample of $P_\ell - P_{\ell-1}$, then the overall cost and variance of the multilevel estimator are:

$$\text{Cost} = \sum_{\ell=0}^{L} N_\ell C_\ell, \quad \text{Variance} = \sum_{\ell=0}^{L} \frac{1}{N_\ell} V_\ell \tag{274}$$

For a fixed variance, we consider the optimization problem:

$$\text{1. Objective:} \quad \text{Minimize} \quad \sum_{\ell=0}^{L} N_\ell C_\ell \tag{275}$$

$$\text{2. Constraint:} \quad \sum_{\ell=0}^{L} \frac{1}{N_\ell} V_\ell = V_{\text{total}} = \varepsilon^2 \tag{276}$$

The cost is minimized by choosing N_ℓ to minimize

$$\sum_{\ell=0}^{L} \left(N_\ell C_\ell + \mu^2 N_\ell^{-1} V_\ell \right) \tag{277}$$

for some value of the Lagrange multiplier μ^2. This gives

$$N_\ell = \mu \sqrt{\frac{V_\ell}{C_\ell}} \tag{278}$$

To achieve an overall variance of ε^2, we require

$$\mu = \varepsilon^{-2} \sum_{\ell=0}^{L} \sqrt{V_\ell C_\ell} \tag{279}$$

and the total computational cost is therefore

$$C = \varepsilon^{-2} \left(\sum_{\ell=0}^{L} \sqrt{V_\ell C_\ell} \right)^2 \tag{280}$$

(a) It is important to know whether the product $V_\ell C_\ell$ increases or decreases with ℓ, i.e., whether C_ℓ increases or not with level faster than V_ℓ decreases

(b) If $V_\ell C_\ell$ increases with ℓ, the dominant contribution comes from $V_L C_L$, then $C_{MLMC} \approx \frac{1}{\varepsilon^2} V_L C_L$

(c) If $V_\ell C_\ell$ decreases with ℓ, the dominant contribution comes from $V_0 C_0$, then $C_{MLMC} \approx \frac{1}{\varepsilon^2} V_0 C_0$

(d) In Monte Carlo we estimate directly from P_L. Assume that the cost of computing P_L is similar to the cost of computing $P_L - P_{L-1}$, i.e., C_L and that $\mathbb{V}[P_L] \approx \mathbb{V}[P_0]$. then the cost of MC is $C_{MC} \approx \frac{1}{\varepsilon^2} V_0 C_L$ which constrast with Multilevel MC.

(e) Assuming the case (b) and (d), then $\frac{C_{MLMC}}{C_{MC}} = \frac{V_L}{V_0}$, which shows that the cost in MLMC is reduced by factor $\frac{V_L}{V_0}$.

(f) Assuming the case (c) and (d), then $\frac{C_{MLMC}}{C_{MC}} = \frac{C_0}{C_L}$, which shows that the cost in MLMC is reduced by factor $\frac{C_0}{C_L}$.

(g) Finally, if the product $V_\ell C_\ell$ does not vary with ℓ, then the total cost is $\varepsilon^{-2} L^2 V_0 C_0 = \varepsilon^{-2} L^2 V_L C_L$.

9.2 Geometric Multilevel Monte Carlo

Before we considered P_L the quantity of interest. In many infinite-dimensional applications, the output P_ℓ on level ℓ is an approximation to a random variable P which can not be simulated exactly.

If Y is an approximation of $\mathbb{E}[P]$, then the MSE reads

$$\text{MSE} \equiv \mathbb{E}\left[(Y - \mathbb{E}[P])^2\right] = \underbrace{(\mathbb{E}[Y] - \mathbb{E}[P])^2}_{\text{bias}} + \underbrace{\mathbb{V}[Y]}_{\text{statistical error}} \tag{281}$$

(a) In Monte Carlo, we assume that P can be simulated and consider $Y = \bar{P}_M$, which has $\mathbb{E}[Y] = \mathbb{E}[P]$ and the first term in the right hand side of (281) vanish.

(b) In General Multilevel Monte Carlo, we assume that $P = P_L$ and P_L can be simulated. Then we consider the Multilevel approximation $Y = \sum_{\ell=0}^{L} Y_\ell$, which has $\mathbb{E}[Y] = \mathbb{E}[P_L]$ and the first term in the right hand side of (281) vanish.

If Y is now the multilevel estimator

$$Y = \sum_{\ell=0}^{L} Y_\ell, \quad Y_\ell = \frac{1}{N_\ell} \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)}\right) \tag{282}$$

with $P_{-1} \equiv 0$, then

$$\mathbb{E}[Y] = \mathbb{E}[P_L], \quad \mathbb{V}[Y] = \sum_{\ell=0}^{L} \frac{1}{N_\ell} V_\ell, \quad V_\ell \equiv \mathbb{V}\left[P_\ell - P_{\ell-1}\right] \tag{283}$$

To get MSE $\leq \varepsilon^2$ in (281), we ensure that

$$\mathbb{V}[Y] \leq \frac{\varepsilon^2}{2}, \quad (\mathbb{E}[P_L] - \mathbb{E}[P])^2 \leq \frac{\varepsilon^2}{2} \tag{284}$$

Idea: Geometric sequence of levels in which the cost increases exponentially with level, while both the weak error $\mathbb{E}[P_L - P]$ and the multilevel correction variance V_ℓ decrease exponentially, leads to the following theorem:

Theorem 12. *Let P denote a random variable, and let P_ℓ denote the corresponding level ℓ numerical approximation.*

If there exist independent estimators Y_ℓ based on N_ℓ Monte Carlo samples, each with expected cost C_ℓ and variance V_ℓ, and positive constants $\alpha, \beta, \gamma, c_1, c_2, c_3$ such that $\alpha \geq \frac{1}{2}\min(\beta, \gamma)$ and

(i) $\mathbb{E}[|P_\ell - P|] \leq c_1 2^{-\alpha\ell}$ *(Weak Error)*

(ii) $\mathbb{E}[Y_\ell] = \begin{cases} \mathbb{E}[P_0], & \ell = 0 \\ \mathbb{E}[P_\ell - P_{\ell-1}], & \ell > 0 \end{cases}$

(iii) $V_\ell \leq c_2 2^{-\beta\ell}$

(iv) $C_\ell \le c_3 2^{\gamma\ell}$

then there exists a positive constant c_4 *such that for any* $\varepsilon < e^{-1}$*, there are values* L *and* N_ℓ *for which the multilevel estimator*

$$Y = \sum_{\ell=0}^{L} Y_\ell \tag{285}$$

has a mean-square-error with bound

$$MSE \equiv \mathbb{E}\left[(Y - \mathbb{E}[P])^2\right] < \varepsilon^2 \tag{286}$$

with a computational complexity C *with bound*

$$\mathbb{E}[C] \le \begin{cases} c_4\varepsilon^{-2}, & \beta > \gamma \\ c_4\varepsilon^{-2}(\log\varepsilon)^2, & \beta = \gamma \\ c_4\varepsilon^{-2-(\gamma-\beta)/\alpha}, & \beta < \gamma \end{cases} \tag{287}$$

9.2.1 Multilevel Monte Carlo Implementation

Algorithm 1 MLMC algorithm

The geometric MLMC algorithm used for the numerical tests in this paper is:

```
start with L = 2, and initial target of N_0 samples on levels ℓ = 0, 1, 2
while extra samples need to be evaluated do
    evaluate extra samples on each level
    compute/update estimates for V_ℓ, ℓ = 0, ..., L
    define optimal N_ℓ, ℓ = 0, ..., L
    test for weak convergence
    if not converged then
        set L := L + 1, and initialise target N_L
    end if
end while
```

(a) In the above algorithm, the equation for the optimal N_ℓ comes from (278) and is

$$N_\ell = \left\lceil 2\,\varepsilon^{-2}\sqrt{V_\ell/C_\ell}\left(\sum_{\ell=0}^{L}\sqrt{V_\ell C_\ell}\right)\right\rceil, \tag{288}$$

where V_ℓ is the estimated variance, and C_ℓ is the cost of an individual sample on level ℓ. This ensures that the estimated variance of the combined multilevel estimator is less than $\frac{1}{2}\varepsilon^2$ (See (284))

(b) The test for weak convergence tries to ensure that $|\mathbb{E}[P - P_L]| < \varepsilon/\sqrt{2}$, to achieve an MSE which is less than ε^2 (See (284)), with ε being a user-specified r.m.s. accuracy. If $\mathbb{E}[P_\ell - P_{\ell-1}] \propto 2^{-\alpha\ell}$ then the remaining error is

$$\mathbb{E}[P - P_L] = \sum_{\ell=L+1}^{\infty}\mathbb{E}[P_\ell - P_{\ell-1}] = \mathbb{E}[P_L - P_{L-1}]\,/\,(2^\alpha - 1) \tag{289}$$

This leads to the convergence test $|\mathbb{E}[P_L - P_{L-1}]|/(2^\alpha - 1) < \varepsilon/\sqrt{2}$, but for robustness, we extend this check to extrapolate from the previous two data points $\mathbb{E}[P_{L-1} - P_{L-2}]$, $\mathbb{E}[P_{L-2} - P_{L-3}]$, and take the maximum over all three as the estimated remaining error.

(c) The conditions of the MLMC theorem assume *a priori* knowledge of the constants c_1 and c_2 governing the weak convergence and the variance convergence as $\ell \to \infty$. These two constants are in effect being estimated in the numerical algorithm described above. In addition, the accuracy of the variance estimate at each level depends on the size of the sample set. If it is small, then this variance estimate may be poor.

The results to be presented later all use the same two routines written in MATLAB:

- `mlmc.m`: driver code which performs the MLMC calculation using an application-specific routine to compute $\sum_n (P_\ell^{(n)} - P_{\ell-1}^{(n)})^p$ for $p = 1, 2$ and a specified number of independent samples;
- `mlmc_test.m`: a program which performs various tests and then calls `mlmc.m` to perform a number of MLMC calculations.

9.2.2 MLMC Driver Routine

The inputs and outputs of the `mlmc.m` function are:

Listing 1: My MATLAB algorithm

```
function [P, Nl, cost] = mlmc(mlmc_l,N0,eps,Lmin,Lmax,
                              alpha,beta,gamma, varargin)

multi-level Monte Carlo estimation
%Outputs
P     = value Y in the theory, approximation of E[P]
Nl    = number of samples at each level
cost  = total cost

%Inputs
N0    = initial number of samples          > 0
eps   = desired accuracy (rms error)       > 0
Lmin  = minimum level of refinement        >= 2
Lmax  = maximum level of refinement        >= Lmin

alpha -> weak error is  O(2^{-alpha*l})
beta  -> variance is    O(2^{-beta*l})
gamma -> sample cost is O(2^{gamma*l})

varargin = optional additional user variables to be passed to mlmc_l

if alpha, beta, gamma are not positive, then they will be estimated
```

The user must supply an application-specific function to perform the calculations to compute Y_ℓ on level ℓ.

$$Y_\ell = \frac{1}{N_\ell} \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)$$

This function has to conform to the following template:

Listing 2: My MATLAB algorithm

```
 %mlmc_l = function for level l estimator

[sums, cost] = mlmc_fn(l,N, varargin)     low-level routine

inputs:  l = level
         N = number of samples
         varargin = optional additional user variables

output: sums(1) = sum(Y)
        sums(2) = sum(Y.^2)
        where Y are iid samples with expected value:
        E[P_0]           on level 0
        E[P_l - P_{l-1}] on level l>0
        cost = cost of N samples
```

The first part of the code initialises various parameters, and then uses `mlmc_l` to generate the initial N_0 samples on each of the first `L=Lmin` levels.

Listing 3: My MATLAB algorithm

```
    function [P, Nl, Cl] = mlmc(mlmc_l,N0,eps,Lmin,Lmax, ...
                            alpha0,beta0,gamma0, varargin)

% check input parameters

% initialisation
  alpha = max(0, alpha0);
  beta  = max(0, beta0);
  gamma = max(0, gamma0);
  theta = 0.25;

  L = Lmin;

  Nl(1:L+1)       = 0;
  suml(1:2,1:L+1) = 0;
  costl(1:L+1)    = 0;
  dNl(1:L+1)      = N0;

  while sum(dNl) > 0
% update sample sums

```

```
22      for l=0:L
23        if dNl(l+1) > 0
24          [sums cost] = mlmc_l(l,dNl(l+1), varargin{:});
25          Nl(l+1)     = Nl(l+1)     + dNl(l+1);
26          suml(1,l+1) = suml(1,l+1) + sums(1);
27          suml(2,l+1) = suml(2,l+1) + sums(2);
28          costl(l+1)  = costl(l+1)  + cost;
29        end
30      end
```

(a) In line 14 `Nl` initialize an array of lenght $L+1$ which will store the N_l values for levels $\ell = 0, ..., L$, that is why of the lenght $L+1$.

(b) In line 15, `suml` is a matrix $2 \times L+1$, thaw will store $\text{sum1}(\ell) = \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)$ (coming from mlmcl-see line 24) in the first row for the first levels $\ell = 0, ..., L\text{min}$. Similarly, for each level, the second row of `suml` stores $\text{sum2}(\ell) = \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)^2$

(c) In line 16, `costl` will store the cost C_ℓ of N_ℓ samples produced by `mlmcl` at each level.

(d) In line 17, `dNl` is the prelimiray $N_\ell, \ell = 0, ..., Lmin$ set to N_0 initially. Observe that we use these N_0 samples for the first $Lmin$ levels in mlmcl (line 24), after that the actual N_ℓ will update and will be stored in `Nl`.

The next part, computes the estimates of interest

Listing 4: My MATLAB algorithm

```
1   % compute absolute average, variance and cost
2   ml = abs(   suml(1,:)./Nl); %ml stores Y_0,...,Y_L, so its
       lenght is initially Lmin+1
3   Vl = max(0, suml(2,:)./Nl - ml.^2); % Vl has lenght Lmin+1
4   Cl = costl./Nl;
```

(a) In line 2, `ml` $= |\mathbb{E}[Y_\ell]|$, which by theorem 12, is an unbiased estimator for $\mathbb{E}[P_\ell - P_{\ell-1}]$. So empirically, we are computing $\frac{1}{N_\ell} \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)$

(b) In line 3, we are estimating $V_\ell = \mathbb{V}[P_\ell - P_{\ell-1}]$

(c) The routine mlmcl, returns cost=cost of N_ℓ samples. So, if we want to know the cost per sample C_ℓ, we compute `costl ./Nl`, as we do in line 4.

From the weak error in Theorem 12, we can see that

$$\begin{aligned} m_\ell &= |\mathbb{E}[Y_\ell]| = |\mathbb{E}[P_\ell - P_{\ell-1}]| = |\mathbb{E}[P_\ell - P] - \mathbb{E}[P_{\ell-1} - P]| \\ &\leq |\mathbb{E}[P_\ell - P]| + |\mathbb{E}[P_{\ell-1} - P]| \leq \mathbb{E}\left[|P_\ell - P|\right] + \mathbb{E}\left[|P_{\ell-1} - P|\right] \\ &\leq c_1 2^{-\alpha\ell} + c_1 2^{-\alpha(\ell-1)} = \left(c_1 + c_1 2^{\alpha}\right) 2^{-\alpha\ell} \end{aligned} \tag{290}$$

Similarly

$$m_{\ell-1} \leq (c_1 + c_1 2^{\alpha})\, 2^{-\alpha(\ell-1)} = 2^{\alpha}\,(c_1 + c_1 2^{\alpha})\, 2^{-\alpha\ell} \tag{291}$$

For the variance we have

$$V_\ell \leq c_2 2^{-\beta\ell} \quad \text{and} \quad V_{\ell-1} \leq 2^{\beta} c_2 2^{-\beta\ell} \tag{292}$$

The next part computes (or in later passes updates) the estimates for $m_\ell \equiv |\mathbb{E}[Y_\ell]|$, and $V_\ell \equiv \mathbb{V}[Y_\ell]$. If the mean and variance are decaying as expected, one would expect $m_\ell = 2^{-\alpha} m_{\ell-1}$, $V_\ell = 2^{-\beta} V_{\ell-1}$. To avoid possible problems from high kurtosis generating poor empirical estimates, the estimates for m_ℓ and V_ℓ are not allowed to decrease by more than factor $\frac{1}{2}$ relative to this anticipated value.

Listing 5: My MATLAB algorithm

```
1      % fix to cope with possible zero values for ml and Vl
2  % (can happen in some applications when there are few samples)
3      for l = 3:L+1
4        ml(l) = max(ml(l), 0.5*ml(l-1)/2^alpha);
5        Vl(l) = max(Vl(l), 0.5*Vl(l-1)/2^beta);
6      end
```

(a) In lines 4 and 5, if m_ℓ or V_ℓ are less than $0.5 * m_\ell(\ell-1)/2^\alpha$ or $0.5 * V_\ell(\ell-1)/2^\beta$, respectively, then take these values instead to preserve the decaying.

If the user has not supplied the values for α, β and/or γ. they are now estimated by linear regression.

Listing 6: My MATLAB algorithm

```
1  % use linear regression to estimate alpha, beta, gamma if not given
2  %
3      A = repmat((1:L)',1,2).^repmat(1:-1:0,L,1);
4
5      if alpha0 <= 0
6        x     = A \ log2(ml(2:end))';
7        alpha = max(0.5,-x(1));
8      end
9
10     if beta0 <= 0
11       x     = A \ log2(Vl(2:end))';
12       beta  = max(0.5,-x(1));
13     end
14
15     if gamma0 <= 0
16       x     = A \ log2(Cl(2:end))';
17       gamma = max(0.5,x(1));
18     end
```

(a) From (290), $m_\ell = \tilde{c} 2^{-\alpha \ell}$, taking $\log_2$ in both sides we get

$$\log_2 m_\ell = \log_2 \tilde{c} - \alpha \ell, \ell = 1, ..., L \tag{293}$$

We have $\log_2 m_\ell$ from the previous steps, so we estimate $\log_2 \tilde{c}$ and α using Linear regression. The solution x in line 4 is $x = [-\alpha, \log_2 \tilde{c}]$. Then we let $\alpha = \max(0.5, -x(1))$. Similarly we estimate β using linear regression and the vector `Vl` previously computed.

Next, the optimal N_ℓ each level is computed using (288), which in turn gives the number of additional samples which will need to be generated in the next pass.

Listing 7: My MATLAB algorithm

```
% set optimal number of additional samples
% ceil rounds each element of X following --> direction in the real
   line to the nearest integer.
Ns  = ceil( sqrt(Vl./Cl)*sum(sqrt(Vl.*Cl)) / ((1-theta)*eps^2) );
dNl = max(0, Ns-Nl);
```

In the final part of the loop, we test for weak convergence as in Algorithm 1. If a new level is added, its variance is estimated by extrapolation, and the optimal number of samples is updated.

Listing 8: My MATLAB algorithm

```
    % if (almost) converged, estimate remaining error and decide
% whether a new level is required
%
    if sum( dNl > 0.01*Nl ) == 0
      % 23/3/18 this change copes with cases with erratic ml values
      range = 0:min(2,L-1);
      rem = max(ml(L+1-range) ./ 2.^(range*alpha)) / (2^alpha - 1);
      %       rem = ml(L+1) / (2^alpha - 1);

      if rem > sqrt(theta)*eps
        if (L==Lmax)
          fprintf(1,'***failed to achieve weak convergence*** \n');
        else
          L        = L+1;
          Vl(L+1) = Vl(L) / 2^beta;
          Cl(L+1) = Cl(L) * 2^gamma;
          Nl(L+1) = 0;
          suml(1:2,L+1) = 0;
          costl(L+1) = 0;

          Ns  = ceil( sqrt(Vl./Cl) * sum(sqrt(Vl.*Cl)) ...
                                   / ((1-theta)*eps^2) );
          dNl = max(0, Ns-Nl);
        end
      end
    end
```

(a) In line 4, `dNl > 0.01*Nl` yields an array with 0 (false) or 1(true) in each level. Obtaining 0 in all levels $\ell's$ means that `Ns-Nl` is less than `0.01*Nl`, so `Nl` is already enough close to the optimal `Ns` for each level. Getting 1 at some level ℓ means that `Ns-Nl` is significant and we proceed to compute the remaining `Ns-Nl` paths at the level were 1 was obtained. This procedure is controled by the condition `while sum(dNl) > 0` at the begining of the function.

(b) If condition in blue above happens, then we proceed to check the weak error. For this we check if the weak error achieves a desired accuracy as explained in 9.2.1 (b)]. If the accuracy is not achieved then we add a new level as in line 14. Since for this new level we want to know the number of sample paths, we use the rate for $V_{L+1} = 2^{-\beta}V_L$ and $C_{L+1} = 2^{\gamma}C_L$ to aproximate V_L and C_L. With these two values we then can cmpute the optimal N_{L+1} as in line 21. Then because of this new level we have `dNl>0`, so we return to the begining of the function to compute the actual values for $\mathbb{E}[P_{L+1} - P_L]$ and $\mathbb{V}[P_{L+1} - P_L]$ now using the N_{L+1} samples.

The final step after the completion of the main loop is to compute the combined multilevel estimator:

Listing 9: My MATLAB algorithm

```
% finally, evaluate multilevel estimator
P = sum(suml(1,:)./N1);
end
```

9.2.3 MLMC Test routine

`mlmc_test.m` first performs a set of calculations using a fixed number of samples on each level of resolution, and produces four plots:

- $\log_2(V_\ell)$ versus level ℓ
- $\log_2(|\mathbb{E}[P_\ell - P_{\ell-1}]|)$ versus level ℓ
- consistency check versus level
- kurtosis versus level

(a) Consistency Check versus level: Suppose $a = \mathbb{E}[P^f_{\ell-1}]$, $b = \mathbb{E}[P^f_\ell]$, and $c = \mathbb{E}[Y_\ell]$ (This comes from our mlmc simulations), then it should be true that $a - b + c \approx 0$. The consistency check verifies that this is true, to within the accuracy one would expect due to Monte Carlo sampling error. This is particularly important when $P^f_\ell \neq P^c_\ell$ because one is using different approximations on the coarse and fine levels.

Since

$$\sqrt{\mathbb{V}[a-b+c]} \;\leq\; \sqrt{\mathbb{V}[a]} + \sqrt{\mathbb{V}[b]} + \sqrt{\mathbb{V}[c]} \tag{294}$$

it computes and plots the ratio

$$\frac{|a-b+c|}{3\left(\sqrt{V_a} + \sqrt{V_b} + \sqrt{V_c}\right)} \tag{295}$$

where V_a, V_b, V_c are empirical estimates for the variances of a, b, c. The probability of this ratio being greater than unity is less than 0.3%. Hence, if it is, it indicates a likely programming error, or a mathematical error in the formulation which violates the crucial identity (299).

(b) kurtosis: The MLMC approach needs a good estimate for $V_\ell = \mathbb{V}[Y_\ell]$. When the number of samples N is large, the standard deviation of the sample variance for a random variable X with zero mean is approximately $\sqrt{(\kappa-1)/N}\ \mathbb{E}[X^2]$ where the kurtosis κ is defined as $\kappa = \mathbb{E}[X^4]/\mathbb{E}[X^2]^2$.

Hence $O(\kappa)$ samples are required to obtain a reasonable estimate for the variance. As well as plotting the kurtosis κ_ℓ, `mlmc_test.m` will give a warning if κ_ℓ is very large, as this is an indication that the empirical variance estimate may be poor. See remark below.

Remark 19. Let X be mean-zero with variance $V = \mathbb{E}[X^2]$ and kurtosis $\kappa = \mathbb{E}[X^4]/\mathbb{E}[X^2]^2$. The standar deviation (sampling spread) of the variance estimator $\hat{V}$ from N samples is

$$\mathrm{sd}(\hat{V}) \approx V\sqrt{\frac{\kappa-1}{N}}, \quad \text{so} \quad \frac{\mathrm{sd}(\hat{V})}{V} \approx \sqrt{\frac{\kappa-1}{N}}. \tag{296}$$

To target a relative s.d. $\le r$ you need

$$N \gtrsim \frac{\kappa-1}{r^2} \quad \Rightarrow \quad N = O(\kappa). \tag{297}$$

This is the "need $O(\kappa)$ samples" statement. **High kurtosis** $\Rightarrow$ heavy tails / rare large values $\Rightarrow$ the sample variance is noisy. You need $N = O(\kappa)$ samples for a *reliable* variance estimate. If κ is large and N modest, $\hat{V}$ can fluctuate a lot (often *underestimating* when rare events aren't seen), so your variance estimate is poor.

Remark 20. Equation (282) gives the natural choice for the multilevel correction estimator Y_ℓ. However, the multilevel theorem allows for the use of other estimators, provided they satisfy the condition ii) which ensures that $\mathbb{E}[Y] = \mathbb{E}[P_L]$. Some use slightly different numerical approximations for the coarse and fine paths in SDE simulations, resulting in the estimator

$$Y_\ell = N_\ell^{-1}\sum_{n=1}^{N_\ell}\left(P_\ell^f\left(\omega^{(n)}\right) - P_{\ell-1}^c\left(\omega^{(n)}\right)\right). \tag{298}$$

Provided we maintain the identity

$$\mathbb{E}\left[P_\ell^f\right] = \mathbb{E}\left[P_\ell^c\right] \tag{299}$$

so that the expectation on level ℓ is the same for the two approximations, then condition ii) is satisfied and no additional bias is introduced, namely,

$$\begin{aligned}
\mathbb{E}[Y_\ell] &= \mathbb{E}\left[P_\ell^f - P_{\ell-1}^c\right] && (300)\\
&= \underbrace{\left(\mathbb{E}\left[P_\ell^f\right] - \mathbb{E}\left[P_\ell^c\right]\right)}_{=0 \text{ by } (299)} + \mathbb{E}\left[P_\ell^c - P_{\ell-1}^c\right] && (301)\\
&= \mathbb{E}\left[P_\ell^c - P_{\ell-1}^c\right]. && (302)
\end{aligned}$$

The next fragment shows the information about the function `mlmc_test`, its inputs and outputs:

Listing 10: My MATLAB algorithm

```
function mlmc_test(mlmc_l, N,L, N0,Eps,Lmin,Lmax, fp, varargin)

multilevel Monte Carlo test routine

[sums cost] = mlmc_l(l,N, varargin)     low-level routine

    inputs:  l = level
             N = number of paths

    output: sums(1) = sum(Pf-Pc)
            sums(2) = sum((Pf-Pc).^2)
            sums(3) = sum((Pf-Pc).^3)
            sums(4) = sum((Pf-Pc).^4)
            sums(5) = sum(Pf)
            sums(6) = sum(Pf.^2)
            cost    = user-defined computational cost

N         = number of samples for convergence tests
L         = number of levels for convergence tests

N0        = initial number of samples for MLMC calcs
Eps       = desired accuracy array for MLMC calcs
Lmin      = minimum number of levels for MLMC calcs
Lmax      = maximum number of levels for MLMC calcs

fp        = file handle for printing to file
varargin = optional additional user variables to be passed to mlmc_l
```

(a) The `mlmc_l` used in the test routine has more outputs in comparison with the `mlmc_l` function in the `mlmc` routine.

Listing 11: My MATLAB algorithm

```
    % first, convergence tests
N = 100*ceil(N/100);   % make N a multiple of 100
del1 = [];
del2 = [];
var1 = [];
var2 = [];
kur1 = [];
chk1 = [];
cost = [];

```

```
11 for l = 0:L
12 %  disp(sprintf('l = %d',l))
13   sums = 0;
14   cst  = 0;
15   parfor j=1:100 %iterations in j will be executed in parallel
16     RandStream.setGlobalStream( ...
17     RandStream.create('mrg32k3a','NumStreams',100,'StreamIndices',j)
          );
18
19     [sums_j cst_j] = mlmc_l(l,N/100, varargin{:});
20     sums = sums + sums_j/N;
21     cst  = cst  + cst_j/N;
22   end
```

(a) Line 15 splits the N samples into **100 equal batches**, and since N is divisible by 100, for each j runs $N/100$ paths.

(b) The next mini example tries to explain what is in lines 16-17. Basically we create 100 sequences or incides that are independents and are used to to generate the random numbers by each worker in the parallel loop. This avoids that different workers use same random numers.

Listing 12: My MATLAB algorithm

```
1          % Creamos 3 secuencias independientes del RNG mrg32k3a.
2 s1 = RandStream.create('mrg32k3a','NumStreams',3,'StreamIndices
     ',1);
3 s2 = RandStream.create('mrg32k3a','NumStreams',3,'StreamIndices
     ',2);
4 s3 = RandStream.create('mrg32k3a','NumStreams',3,'StreamIndices
     ',3);
5
6 % Usamos la secuencia 1 como fuente global y tomamos 2 numeros
7 RandStream.setGlobalStream(s1);  a = rand(1,2); % = (u1^1, u2
     ^1)
8
9 % Cambiamos a la secuencia 2 y tomamos 2 numeros
10 RandStream.setGlobalStream(s2);  b = rand(1,2); % = (u1^2, u2
     ^2)
11
12 % Volvemos a la secuencia 1 y seguimos donde quedo su estado
13 RandStream.setGlobalStream(s1);  c = rand(1,2);   % = (u3^1, u4
     ^1)
14
15 % Re-creamos la secuencia 1 desde el inicio y comprobamos
     reproducibilidad
16 s1b = RandStream.create('mrg32k3a','NumStreams',3,'
     StreamIndices',1);
```

```
17 RandStream.setGlobalStream(s1b); d = rand(1,2);  % = (u1^1, u2
     ^1)  igual que 'a'
```

(c) `mlmc_l` returns *totals* over that batch:

$$\text{sums\_j} = \left[\Sigma(P^f - P^c),\ \Sigma(P^f - P^c)^2,\ \Sigma(P^f - P^c)^3,\ \Sigma(P^f - P^c)^4,\ \Sigma P^f,\ \Sigma(P^f)^2\right], \quad \text{cst\_j} = \text{total cost}.$$

The lines `sums += sums_j/N` and `cst += cst_j/N` accumulate *global averages*: after the `parfor`,

$$\text{sums} = \frac{1}{N}\sum_{n=1}^{N}\left[Y,\ Y^2,\ Y^3,\ Y^4,\ P^f,\ (P^f)^2\right],$$

$$\text{cst} = \frac{1}{N}\sum \text{cost\_per\_path}.$$

Listing 13: My MATLAB algorithm

```
1     if (l==0)
2     kurt = 0.0;
3   else
4     kurt = (      sums(4)                ...
5               - 4*sums(3)*sums(1)        ...
6               + 6*sums(2)*sums(1)^2      ...
7               - 3*sums(1)*sums(1)^3 )    ...
8             / (sums(2)-sums(1)^2)^2;
9   end
10
11  cost = [cost cst];
12 del1 = [del1 sums(1)]; % E[Pf-Pc] = m_l
13 del2 = [del2 sums(5)]; % E[Pf]
14 var1 = [var1 sums(2)-sums(1)^2 ]; % Var(Pf-Pc) = V_l
15 var2 = [var2 sums(6)-sums(5)^2 ]; % Var(Pf)
16 var2 = max(var2, 1e-10); % avoid zero
17 kur1 = [kur1 kurt];
```

(a) In line 4, computes

$$\kappa_\ell = \frac{\mathbb{E}[Y_\ell^4] - 4\mu\mathbb{E}[Y_\ell^3] + 6\mu^2\mathbb{E}[Y_\ell^2] - 3\mu^4}{\left(\mathbb{E}[Y_\ell^2] - \mu^2\right)^2}, \quad \mu = \mathbb{E}[Y_\ell], \tag{303}$$

i.e., the kurtosis of Y_ℓ.

Listing 14: My MATLAB algorithm

```
1       if l==0
2         check = 0;
3       else
4         check = abs(        del1(l+1)   +       del2(l)   -       del2(l+1))
            ...
5           /     ( 3.0*(sqrt(var1(l+1)) + sqrt(var2(l)) + sqrt(var2(l+1))
              )/sqrt(N));
6       end
7       chk1 = [chk1 check];
8
9       PRINTF2(fp,'%2d  %11.4e %11.4e  %.3e  %.3e  %.2e  %.2e  %.2e \n',
          ...
10              l,del1(l+1),del2(l+1),var1(l+1),var2(l+1),kur1(l+1),chk1(l
                  +1),cst);
11  end (end of loop for l=0:L)
12  % print out a warning if kurtosis or consistency check looks bad
13  if ( kur1(end) > 100.0 )
14    PRINTF2(fp,'\n WARNING: kurtosis on finest level = %f \n',kur1(end
        ));
15    PRINTF2(fp,' indicates MLMC correction dominated by a few rare
        paths; \n');
16    PRINTF2(fp,' for information on the connection to variance of
        sample variances ,\n');
17    PRINTF2(fp,' see http://mathworld.wolfram.com/
        SampleVarianceDistribution.html\n\n');
18  end
19
20  if ( max(chk1) > 1.0 )
21    PRINTF2(fp,'\n WARNING: maximum consistency error = %f \n',max(
        chk1));
22    PRINTF2(fp,' indicates identity E[Pf-Pc] = E[Pf] - E[Pc] not
        satisfied; \n');
23    PRINTF2(fp,' to be more certain , re-run mlmc_test with larger N \n
        \n');
24  end
```

(a) Line 4 computes the ration (295). For each level in the loop, if this ration is greater than 1, then there is a possible problem in the coupling or implementation of the estimator al level ℓ. At the end this causes a problem with condition ii) of the theorem, and consequently, the telecopic sum.

(b) In line 13, we check kurtosis on the finest level, a warning is preinted out if kurtosis is high.

The next part estimates α, β, γ, applying a linear regression using the values for each level computed before.

Listing 15: My MATLAB algorithm

```
 % use linear regression to estimate alpha, beta and gamma
L1 = 2;
L2 = L+1;
pa    = polyfit(L1:L2,log2(abs(del1(L1:L2))),1);  alpha = -pa(1);
pb    = polyfit(L1:L2,log2(abs(var1(L1:L2))),1);  beta  = -pb(1);
pg    = polyfit(L1:L2,log2(abs(cost(L1:L2))),1);  gamma =  pg(1);

PRINTF2(fp,'\n
   ****************************************************\n');
PRINTF2(fp,'*** Linear regression estimates of MLMC parameters ***\n
   ');
PRINTF2(fp,'****************************************************\n
   ');
PRINTF2(fp,'\n alpha = %f  (exponent for MLMC weak convergence)\n',
   alpha);
PRINTF2(fp,' beta  = %f  (exponent for MLMC variance) \n',beta);
PRINTF2(fp,' gamma = %f  (exponent for MLMC cost) \n',gamma);
```

Next, we make a complexity test, for that we reset the random numbers generators and set the values for α, β, and γ.

Listing 16: My MATLAB algorithm

```
    % reset random number generators

reset(RandStream.getGlobalStream);
spmd
  RandStream.setGlobalStream( ...
  RandStream.create('mrg32k3a','NumStreams',numlabs,'StreamIndices',
     labindex));
end

alpha = max(alpha,0.5);
beta  = max(beta,0.5);
theta = 0.25;
```

In the final part, we computes our estimates using mlmc routine:

Listing 17: My MATLAB algorithm

```
    for i = 1:length(Eps)
  eps = Eps(i);
  [P, Nl, Cl] = mlmc(mlmc_l,N0,eps,Lmin,Lmax, alpha,beta,gamma, ...
                     varargin{:});
  mlmc_cost = sum(Nl.*Cl);
  std_cost  = var2(min(end,length(Nl)))*Cl(end) / ((1.0-theta)*eps
     ^2);

  PRINTF2(fp,'%.3e %10.3e  %.3e  %.3e  %7.2f ', ...
```

```
9             eps, P, mlmc_cost, std_cost, std_cost/mlmc_cost);
10    PRINTF2(fp,'%9d',Nl);
11    PRINTF2(fp,'\n');
12  end
13
14  PRINTF2(fp,'\n');
15
16  end
```

(a) `eps`: target accuracy ε for MSE.

`value`: MLMC estimate $P \approx \mathbb{E}[\text{payoff}]$ returned by `mlmc(...)`.

`mlmc_cost`: total MLMC work

$$\text{mlmc\_cost} = \sum_{\ell} N_\ell C_\ell \tag{304}$$

where N_ℓ = samples per level, C_ℓ = cost per sample on level ℓ.

`std_cost`: **single-level Monte Carlo cost on the finest level used**, to reach variance target $(1-\theta)\varepsilon^2$ (See remark below)

$$\text{std\_cost} = \frac{\text{Var}(P_{L^*}^f)\, C_{L^*}}{(1-\theta)\varepsilon^2} \tag{305}$$

coded as `var2(min(end,length(Nl)))*Cl(end)/((1.0-theta)*eps^2)`.

`savings`: ratio std_cost/mlmc_cost (> 1 means MLMC is cheaper than standard MC).

`N_l`: the vector N_ℓ (optimal MLMC sample allocation across levels).

Remark 21. Crude MC on the finest level used by MLMC (call it L^*).

$$\bar{P}_N = \frac{1}{N}\sum_{i=1}^{N} P_{L^*}^{(i)}. \tag{306}$$

Its variance is

$$\text{Var}(\bar{P}_N) = \frac{\text{Var}(P_{L^*})}{N}. \tag{307}$$

In the test they use an **MSE split**:

$$\text{MSE} = \text{bias}^2 + \text{Var}(\bar{P}_N) \le \varepsilon^2, \tag{308}$$

allocating

$$\text{bias}^2 \le \theta\varepsilon^2, \quad \text{Var}(\bar{P}_N) \le (1-\theta)\varepsilon^2,$$

with $\theta = 0.25$ in the code. From $\text{Var}(\bar{P}_N) \le (1-\theta)\varepsilon^2$ we get the **required sample size** for crude MC:

$$N_{\text{std}} \ge \frac{\text{Var}(P_{L^*})}{(1-\theta)\varepsilon^2}. \tag{309}$$

If the *cost per sample* on level L^* is C_{L^*}, the total crude-MC cost is

$$\text{std\_cost} = N_{\text{std}} C_{L^*} \approx \frac{\text{Var}(P_{L^*})\, C_{L^*}}{(1-\theta)\varepsilon^2}. \tag{310}$$

9.3 Multilevel Monte Carlo Path Simulations

Consider the following multidimensional SDE

$$\mathrm{d}S(t) = a(S,t)\,\mathrm{d}t + b(S,t)\,\mathrm{d}W(t), \quad 0 < t < T, \tag{311}$$

with given initial data S_0. We want to compute $\mathbb{E}f(S(T))$, where $f(S)$ is a scalar function with a uniform Lipschitz bound, i.e.,

$$|f(U) - f(V)| \le c\|U - V\| \quad \forall\, U, V. \tag{312}$$

A simple Euler discretisation of this SDE with time-step h is

$$\hat{S}_{n+1} = \hat{S}_n + a(\hat{S}_n, t_n)h + b(\hat{S}_n, t_n)\Delta W_n, \tag{313}$$

and the simplest estimate for $\mathbb{E}[f(S_T)]$ is the mean of the payoff values $f(\hat{S}^{(i)}_{T/h})$, from N independent path simulations (Crude Monte Carlo):

$$\hat{Y} = N^{-1} \sum_{i=1}^{N} f(\hat{S}^{(i)}_{T/h}). \tag{314}$$

It is well established that, provided $a(S,t)$ and $b(S,t)$ satisfy certain conditions the expected mean-square error (MSE) in the estimate $\hat{Y}$ is asymptotically of the form

$$\text{MSE} \approx \underbrace{c_2 h^2}_{\text{bias}} + \underbrace{c_1 N^{-1}}_{\text{variance}}. \tag{315}$$

where c_1, c_2 are positive constants.

(a) See (253), the first terms is the square of the $O(h)$ bias introduced by the Euler discretisation (weak error) (249), and the second term is the variance in $\hat{Y}$ due to the Monte Carlo sampling (255)

(b) To make the MSE $O(\varepsilon^2)$ in (315), so that the RSME is $O(\varepsilon)$, requires that $N = O(\varepsilon^{-2})$ and $h = O(\varepsilon)$. Since the paths are independent, the total cost is $C \approx n \times N$, where $n = T/h = O(\varepsilon^{-1})$,hence the computational complexity (cost) is $O(\varepsilon^{-3})$

(c) The computational complexity $O(\varepsilon^{-3})$ can be reduced to $O(\varepsilon^{-2}(\log\varepsilon)^2)$ through the use of a multilevel MC method that reduces the variance, leaving unchanged the bias due to the Euler discretisation.

9.3.1 MLMC Method

- Consider Monte Carlo path simulations with different timesteps $h_\ell = M^{-\ell}T$, $\ell = 0, 1, \ldots, L$.
- For a given Brownian path $W(t)$, let

$$P = f(S(T)), \quad \text{and} \quad \hat{S}_{\ell,M^\ell} \approx S(T),\ \hat{P}_\ell \approx f(S(T)) = P \tag{316}$$

  using a numerical discretisation with timestep h_ℓ.

It is clearly true that

$$\mathbb{E}[\hat{P}_L] = \mathbb{E}[\hat{P}_0] + \sum_{\ell=1}^{L} \mathbb{E}[\hat{P}_\ell - \hat{P}_{\ell-1}]. \tag{317}$$

(a) The multilevel method independently estimates each of the expectations on the right-hand of (317) side in a way that minimises the computational complexity.

Let $\hat{Y}_0$ be an estimator for $\mathbb{E}[\hat{P}_0]$ using N_0 samples, and let $\hat{Y}_\ell$ for $\ell > 0$ be an estimator for $\mathbb{E}[\hat{P}_\ell - \hat{P}_{\ell-1}]$ using N_ℓ paths. Consider the estimator as the mean of N_ℓ independent samples, which for $\ell > 0$ is

$$\hat{Y}_\ell = N_\ell^{-1} \sum_{i=1}^{N_\ell} \left(\hat{P}_\ell^{(i)} - \hat{P}_{\ell-1}^{(i)} \right). \tag{318}$$

The key point here is that the quantity $\hat{P}_\ell^{(i)} - \hat{P}_{\ell-1}^{(i)}$ comes from two discrete approximations with different timesteps but the same Brownian path. This is easily implemented by first constructing the Brownian increments for the simulation of the discrete path leading to the evaluation of $\hat{P}_\ell^{(i)}$, and then summing them in groups of size M to give the discrete Brownian increments for the evaluation of $\hat{P}_{\ell-1}^{(i)}$. For a more detailed explanation see Remark 27.

The variance of the estimator $\hat{Y}_\ell$ of $\mathbb{E}[P_\ell - P_{\ell-1}]$ is

$$\mathbb{V}[\hat{Y}_\ell] = \frac{1}{N_\ell} V_\ell, \quad V_\ell = \mathbb{V}[P_\ell - P_{\ell-1}] \tag{319}$$

The variance of the combined estimator $\hat{Y} = \sum_{\ell=0}^{L} \hat{Y}_\ell$ is

$$\mathbb{V}[\hat{Y}] = \sum_{\ell=0}^{L} \frac{1}{N_\ell} V_\ell \tag{320}$$

The computational cost, if one ignores the asymptotically negligible cost of the final payoff evaluation, is proportional to (for details see Remark 28)

$$\sum_{\ell=0}^{L} N_\ell \, h_\ell^{-1}. \tag{321}$$

Treating the N_ℓ as continuous variables, the variance is minimized for a fixed computational cost by choosing

$$N_\ell \propto \sqrt{V_\ell h_\ell} \tag{322}$$

The above analysis holds for any value of L. We now assume that $L \gg 1$, and consider the behaviour of V_ℓ as $\ell \to \infty$. In the particular case of the Euler discretisation and the Lipschitz payoff function, provided $a(S,t)$ and $b(S,t)$ satisfy certain conditions there is $O(h)$ weak convergence and $O(h^{1/2})$ strong convergence. Hence, as $\ell \to \infty$,

$$\mathbb{E}[\hat{P}_\ell - P] = O(h_\ell) \tag{323}$$

$$\mathbb{E}[\|\hat{S}_{\ell,M^\ell} - S(T)\|^2] = O(h_\ell) \tag{324}$$

From the Lipschitz property (312), it follows that

$$\mathbb{V}[\hat{P}_\ell - P] \leq {}^2\mathbb{E}[(\hat{P}_\ell - P)^2] \leq c^2\mathbb{E}[\|\hat{S}_{\ell,M^\ell} - S(T)\|^2] \tag{325}$$

Combining this with (324) gives $\mathbb{V}[\hat{P}_\ell - P] = O(h_\ell)$. Furthermore,

$$(\hat{P}_\ell - \hat{P}_{\ell-1}) = (\hat{P}_\ell - P) - (\hat{P}_{\ell-1} - P) \tag{326}$$

$$\Rightarrow \quad \mathbb{V}[\hat{P}_\ell - \hat{P}_{\ell-1}] \leq \left(\mathbb{V}[\hat{P}_\ell - P]^{1/2} + \mathbb{V}[\hat{P}_{\ell-1} - P]^{1/2}\right)^2 \text{ (see Remark 29)} \tag{327}$$

Hence, for the simple estimator (318), the single sample variance

$$V_\ell = O(h_\ell) \tag{328}$$

and from (322), $N_\ell \propto h_\ell$. Setting $N_\ell = O(\varepsilon^{-2}Lh_\ell)$, the variance of $\hat{Y}$ in (320) is $O(\varepsilon^2)$.

If L is now chosen such that

$$L = \frac{\log \varepsilon^{-1}}{\log M} + O(1), \tag{329}$$

as $\varepsilon \to 0$, then $h_L = M^{-L} = O(\varepsilon)$ (see Remar 30), and so the bias error $\mathbb{E}[\hat{P}_L - P]$ is $O(\varepsilon)$, due to (323). We recall that for any unbiased/biased Monte-Carlo estimator

$$\text{MSE}(\hat{Y}) = \underbrace{\left(\mathbb{E}[\hat{Y}] - \mathbb{E}[P]\right)^2}_{\text{bias}} + \underbrace{\mathbb{V}[\hat{Y}]}_{\text{statistical error}} \tag{330}$$

Consequently, since $\mathbb{E}[\hat{Y}] = \mathbb{E}[P_L]$, we obtain an MSE that is $O(\varepsilon^2)$, with a computational complexity that is

$$O(\varepsilon^{-2}L) = O(\varepsilon^{-2}(\log \varepsilon)^2). \tag{331}$$

[2] $\text{Var}(Z) = \mathbb{E}[Z^2] - \left(\mathbb{E}[Z]\right)^2 \leq \mathbb{E}[Z^2]$ since $\left(\mathbb{E}[Z]\right)^2 \geq 0$.

9.3.2 Geometric Brownian Motion

We consider the GBM SDE under the risk-neutral measure

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t, \qquad S_0 = K = 100, \tag{332}$$

with $r = 0.05$, $\sigma = 0.2$, and maturity $T = 1$. We want to estimate the price of an European call option, more precisely, we want to estimate P below

$$f(S_T) = (S_T - K)^+, \qquad P = e^{-rT}\,\mathbb{E}[f(S_T)]\,. \tag{333}$$

We want to approximate P using MLMC. To this end, we first approximate S_T using Euler Maruyama discretization as follows:

$$M = 4, \quad n_f = 4^\ell, \quad h_f = 4^{-\ell}T = \frac{T}{nf}, \quad t_i^f = ih_f, \quad i = 0, 1, \ldots, n_f. \tag{334}$$

Similraly, for the course level we have

$$M = 4, \quad n_c = 4^{\ell-1} = \frac{n_f}{4}, \quad h_c = 4^{-(\ell-1)}T = \frac{T}{n_c}, \quad t_j^c = jh_{\ell-1} = 4jh_c, \quad j = 0, 1, \ldots, n_c \tag{335}$$

Hence the Euler scheme is expressed as

$$S_{n+1}^{(\ell)} = S_n^{(\ell)} + rS_n^{(\ell)}h_\ell + \sigma S_n^{(\ell)}\Delta W_n^{(\ell)}, \quad \Delta W_n^{(\ell)} \sim N(0, h_\ell), n = 0, ..., 4^\ell - 1 \tag{336}$$

For the level $\ell - 1$, we have

$$S_{k+1}^{(\ell-1)} = S_k^{(\ell-1)} + rS_k^{(\ell-1)}h_{\ell-1} + \sigma S_k^{(\ell-1)}\Delta W_k^{(\ell-1)}, \quad \Delta W_k^{(\ell-1)} \sim \mathcal{N}(0, h_{\ell-1}), \quad k = 0, \ldots, 4^{\ell-1} - 1. \tag{337}$$

where the Brownian increments are obtained usuming the Brownina increments of the fine level ℓ

$$\Delta W_k^{(\ell-1)} = \sum_{m=1}^{4} \Delta W_{4(k-1)+m}^{(\ell)}. \tag{338}$$

Listing 18: My MATLAB algorithm

```
    function [sums, cost] = opre_l(l,N, option)

M = 4;

T    = 1;
r    = 0.05;
sig  = 0.2;
K    = 100;

nf = M^l;
nc = nf/M;

hf = T/nf;
hc = T/nc;

sums(1:6) = 0;
```

Next we continue defining the estimator at level ℓ

Listing 19: My MATLAB algorithm

```
for N1 = 1:10000:N %N1 = [1,10001,20001,30001,.,(1 + 10000*k)leq N].
  N2 = min(10000,N-N1+1); %N-N1+1: Cuantos paths quedandesde N1
      hasta N
    % N2 es: 10000 si quedan >= 10000 paths, el resto si quedan
       menos

    X0 = K;    % S0 = K = 100

    Xf = X0*ones(1,N2); % fine  paths at t=0 (N2 paths in this batch
    Xc = Xf;            % coarse paths at t=0
```

Listing 20: My MATLAB algorithm

```
    if l==0
      dWf = sqrt(hf)*randn(1,N2);
      Xf  = Xf + r*Xf*hf + sig*Xf.*dWf;
```

(a) For $\ell = 0$,$h_f = T$ (since $n_f = 1$), so you take **one** Euler–Maruyama step to reach $S_T^{(\ell=0)}$. For European, only `Xf` at the end is needed.

Listing 21: My MATLAB algorithm

```
1  for n = 1:nc   % nc coarse steps, each spans M fine steps
2    dWc = zeros(1,N2);
3    for m = 1:M                  % M = 4 fine substeps per coarse step
4      dWf = sqrt(hf)*randn(1,N2);   % dW_f -> N(0, hf)
5      dWc = dWc + dWf;   % aggregate: dW_c = sum dW_f -> N(0, hc)
6      Xf  = Xf + r*Xf*hf + sig*Xf.*dWf;   % fine Euler update
7    end
8    Xc = Xc + r*Xc*hc + sig*Xc.*dWc;      % one coarse Euler update
9  end
10 Pf = max(0,Xf-K);
11 Pc = max(0,Xc-K);
```

(a) For the case $\ell \geq 1$, line 5 implements the MLMC coupling:

$$\Delta W_c = \sum_{m=1}^{M} \Delta W_f \sim \mathcal{N}(0, h_c), \tag{339}$$

so fine and coarse share the same Brownian motion when viewed at coarse times. After the loop you have terminal values $Xf = S_T^{(\ell)}$ and $Xc = S_T^{(\ell-1)}$.

Listing 22: My MATLAB algorithm

```
    Pf = exp(-r*T)*Pf;
  Pc = exp(-r*T)*Pc;

  if l==0
    Pc=0;
  end

  sums(1) = sums(1) + sum(Pf-Pc);
  sums(2) = sums(2) + sum((Pf-Pc).^2);
  sums(3) = sums(3) + sum((Pf-Pc).^3);
  sums(4) = sums(4) + sum((Pf-Pc).^4);
  sums(5) = sums(5) + sum(Pf);
  sums(6) = sums(6) + sum(Pf.^2);
end
```

The cost at level ℓ is computed as follows

Listing 23: My MATLAB algorithm

```
    cost = N*nf;   % cost defined as number of fine timesteps times
       the number of samples
```

Remark 22. Here $h_\ell = 4^{-\ell} h_0$, then this gives $\alpha = 2$, $\beta = 2$ and $\gamma = 2$. Alternatively, if $h_\ell = 2^{-\ell} h_0$, then $\alpha = 1$, $\beta = 1$ and $\gamma = 1$. In either case, Theorem 1 gives the complexity to achieve a root-mean-square error of ε to be $\mathcal{O}(\varepsilon^{-2}(\log \varepsilon)^2)$.

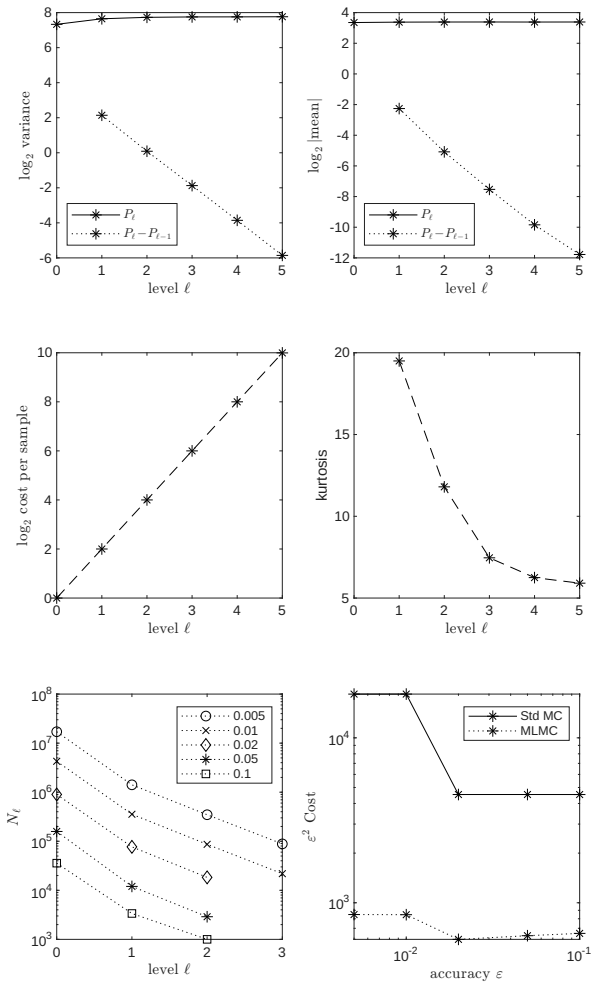

Figure 9: Numerical results for a European call option using the Euler-Maruyama discretisation of the GBM SDE.

Figure 9 shows results for a financial call option with a single underlying asset satisfying Geometric Brownian Motion SDE (332). From the weak error of the Euler-Maruyama discretization (323) and making a similar analyisis as in (290) we obtain that

$$\mathbb{E}[P_\ell - P_{\ell-1}] = O(h_\ell)$$

Also we have from (328) thah

$$V_\ell = \mathbb{V}[\hat{P}_\ell - \hat{P}_{\ell-1}] = O(h_\ell)$$

(a) The top two plots show the variance and absolute mean value for both P_ℓ and $P_\ell - P_{\ell-1}$. The line for $\log_2 \mathbb{V}[P_\ell - P_{\ell-1}]$ in the top left plot has a slope of approximately -2 (If we take $\log_4$ the slope should be approx -1) corresponding to a variance proportional to h_ℓ, as expected. The line for $\log_2 |\mathbb{E}[P_\ell - P_{\ell-1}]|$ also has a slope of approximately -2, implying an $\mathcal{O}(h_\ell)$ weak convergence.

(b) The middle two plots show the cost per level. As we have did in the implementation $C_\ell = O(n_f) = O(M^\ell)$, plotting $\log_2 C_\ell$, the slope is approx 2. The other figure is the kurtosis, as discussed before. This indicate the kurtosis is actually improving slightly with level.

(c) The bottom two plots have the results from 5 separate MLMC calculations, each with a different specified accuracy ε to be achieved. The bottom left plot shows the number of samples which is used for each level of the MLMC computation. The number of levels increases as ε decreases, as described in the mlmc Algorithm to achieve the required weak error. The number of samples also varies with level, with many more on the coarsest level with just one timestep. On the *finer* levels, the optimal allocation of computational effort leads to

$$N_\ell \propto \sqrt{V_\ell / C_\ell} \;\approx\; .$$

The bottom right plot displays the total computational cost C, multiplied by ε^2. If C were approximately proportional to ε^{-2} then $\varepsilon^2 C$ would be approximately constant. Indeed this is what is seen, with the slight increase as ε decreases being due to the $|\log \varepsilon|^2$ term (287). For comparison, the bottom right plot also displays the cost using the standard Monte Carlo algorithm on the finest level used by the MLMC algorithm. The cost ratio is $5 - 12$, illustrating the benefits provided by the MLMC algorithm.

10 Appendix

Definition 11. Let $(\mathbf{X}_n)_{n\in\mathbb{N}}$ be a sequence of random vectors in $\mathbb{R}^m$ and let $\mathbf{X}$ be a random vector in $\mathbb{R}^m$. We say that $(\mathbf{X}_n)_{n\in\mathbb{N}}$ *converges in distribution* to $\mathbf{X}$ if

$$\lim_{n\to\infty} F_{\mathbf{X}_n}(\mathbf{t}) = F_{\mathbf{X}}(\mathbf{t})$$

for all $\mathbf{t} \in \mathbb{R}^m$ that are continuity points of $F_{\mathbf{X}}$. In this case, we write

$$X_n \xrightarrow{d} \mathbf{X} \quad \text{as } n \to \infty.$$

Remark 23. Note that convergence in distribution only depends on the distributions of $(X_n)_{n\in\mathbb{N}}$ and $\mathbf{X}$, whence they do not need to be defined on the same probability space. This is in contrast to almost sure convergence, convergence in probability and L^p-convergence, where all involved random variables must have the same underlying probability space. Sometimes convergence in distribution is also called *weak convergence*.

Theorem 13. *Let $(\mathbf{X}_n)_{n\in\mathbb{N}}$ be a sequence of random vectors in $\mathbb{R}^m$ and let $\mathbf{X}$ be a random vector in $\mathbb{R}^m$. Then, the following statements are equivalent:*

(a) $(\mathbf{X}_n)_{n\in\mathbb{N}}$ converges in distribution to $\mathbf{X}$.

(b) For each bounded continuous function $f : \mathbb{R}^m \to \mathbb{R}$,

$$\mathbb{E}[f(\mathbf{X}_n)] \to \mathbb{E}[f(\mathbf{X})] \quad \textit{as} \quad n \to \infty. \tag{340}$$